

웹 기술 이론 2장

CSS — 캐스케이드와 특정도

누가 이기는가, 상속은 어디까지, 왜 CSS가 예상대로 안 먹는가

한신대학교 AI·SW계열 · 웹프로그래밍
학습사이트 hs-web.dreamitbiz.com

학습 목표

- ① 캐스케이드의 세 판정 기준(중요도·특정도·순서)을 순서대로 적용할 수 있다.
- ② 특정도를 계산해 어떤 규칙이 이길지 예측할 수 있다.
- ③ 상속되는 속성과 안 되는 속성을 구분할 수 있다.
- ④ !important가 왜 문제인지, 대신 무엇을 해야 하는지 설명할 수 있다.
- ⑤ "CSS가 안 먹는" 상황을 체계적으로 진단할 수 있다.

핵심 키워드

캐스케이드

특정도

상속

!important

원본 순서

초기값

inherit

진단

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

이론 2장 · 1

CSS의 C — Cascading

CSS는 Cascading Style Sheets 입니다.

CSS의 C — Cascading

- 1 Cascade는 "계단식 폭포"라는 뜻으로, 여러 곳에서 온 스타일 규칙이 위에서 아래로 흘러내리며 하나의 최종 값으로 정리되는 과정을 가리킵니다.
- 2 이 과정을 모르면 CSS는 "가끔 되고 가끔 안 되는 것"처럼 느껴집니다.
- 3 반대로 이 장을 이해하면 "왜 안 먹지?"를 추측이 아니라 판정으로 해결할 수 있게 됩니다.

이론 2장 · 2

누가 이기는가 — 세 단계 판정

같은 요소의 같은 속성에 여러 규칙이 걸릴 때, 브라우저는 아래 순서로 판정합니다.

누가 이기는가 — 세 단계 판정

순위	기준	설명
①	출처와 중요도	!important 가 붙은 것이 이깁니다
②	특정도	더 구체적으로 지목한 선택자가 이깁니다
③	원본 순서	특정도가 같으면 나중에 쓴 것이 이깁니다

앞 단계에서 결판이 나면 뒤 단계는 보지 않습니다.

이론 2장 · 3

특정도 계산

특정도는 흔히 (a, b, c) 세 자리 숫자로 표기합니다.

특정도 계산

종류	점수	예
인라인 style 속성	1000	<p style="color:red">
아이디	100	#lead
클래스 · 가상클래스 · 속성 선택자	10	.note · :hover · [type="text"]
태그 · 가상요소	1	p · ::before
전체 선택자 * · 결합자 > + ~	0	점수에 영향 없음

편의상 100·10·1점으로 계산해도 대부분의 경우 결과가 같습니다.

계산해 보기

```
1  p                /* 1 */
2  .note            /* 10 */
3  p.note           /* 1 + 10 = 11 */
4  ul li a          /* 1 + 1 + 1 = 3 */
5  .card .title     /* 10 + 10 = 20 */
6  #lead            /* 100 */
7  #lead .title     /* 100 + 10 = 110 */
8  p:hover           /* 1 + 10 = 11 */
9  .item:not(.active) /* 10 + 10 = 20 ← :not 자체는 0점, 안의 것만 센다 */
```

특정도 계산

- 1 **:not()** 안의 선택자는 점수에 포함됩니다.
- 2 **:not** 자체는 0점이지만 괄호 안의 **.active**가 10점으로 더해집니다.
- 3 **:is()·:where()**도 알아 두면 좋습니다 — **:where()**는 안의 것까지 0점으로 만들어 주므로 덮어쓰기 쉬운 기본 스타일을 만들 때 유용합니다.

클래스 100개도 아이디 하나를 못 이깁니다

```
1 .a.b.c.d.e.f.g.h.i.j    /* (0,10,0) — 클래스 10개 */
2 #x                      /* (1,0,0) — 아이디 하나 ← 이깁니다 */
3
4 /* 100점 단위가 다르면 아래 자릿수는 아무리 커도 소용없습니다.
5    그래서 아이디로 스타일을 주면 나중에 이기기가 매우 어려워집니다. */
```

이론 2장 · 4

상속

어떤 속성은 자식에게 자동으로 물려집니다.

상속

상속됨	상속 안 됨
color	background
font-family · font-size · font-weight	margin · padding
line-height · letter-spacing	border
text-align · text-indent	width · height
visibility · cursor	display · position
list-style	overflow

모든 요소에 글꼴을 일일이 지정하지 않아도 되는 것은 상속 덕분입니다.

상속

- 1 기준은 대체로 "글자에 관한 것은 상속, 상자에 관한 것은 비상속" 입니다.
- 2 상자 속성이 상속되면 중첩된 div마다 테두리가 그려지는 이상한 일이 벌어질 테니까요.

상속을 강제하거나 끄기

```
1  .child {
2    color: inherit; /* 부모 값을 강제로 물려받는다 */
3    border: initial; /* 브라우저 초기값으로 되돌린다 */
4    margin: unset; /* 상속 속성이면 inherit, 아니면 initial */
5    all: revert; /* 브라우저 기본 스타일로 되돌린다 */
6  }
7
8  /* 자주 쓰는 곳: 버튼 · 입력칸의 글꼴은 상속되지 않으므로 명시해야 합니다 */
9  button, input, select, textarea { font-family: inherit; font-size: inherit; }
```

상속

- 1 상속이 만드는 함정.
- 2 `h1, h2, h3 { color: #111 }` 같은 전역 규칙이 있으면, 어두운 배경 위에 올린 제목이 검은 글씨로 묻혀 안 보입니다.
- 3 베이스 스타일을 재사용할 때 반복해서 밟는 지점이라, 어두운 배경 컴포넌트에서는 색을 명시하는 것이 원칙입니다.

이론 2장 · 5

!important 를 쓰지 않는 이유

!important 는 특정도를 통째로 건너뛰고 이깁니다.

악순환

```
1 .btn { background: blue !important; }
2
3 /* 나중에 특정 화면에서만 빨갱게 하고 싶다 */
4 .page .btn { background: red; }           /* 안 먹는다 */
5 .page .btn { background: red !important; } /* 또 important를 써야 한다 */
6
7 /* 그다음에는? 무엇이 이기는지 아무도 모르는 CSS가 됩니다. */
```

당장은 편하지만 되돌릴 수 없는 선택입니다.

important 대신 해야 할 것

● 정리

- 1 특정도를 한 단계 올린다**
.btn 대신 .card .btn.
- 2 순서를 조정한다**
이겨야 하는 규칙을 파일 뒤로 옮깁니다.
- 3 클래스를 하나 더 붙인다**
.btn.btn--danger (11점 → 20점).
- 4 예외: 남의 라이브러리 스타일을 덮어써야 할 때, 그리고 [hidden] { display: none !important } 처럼 반드시 이겨야 하는 유틸리티에는 정당하게 씁니다.**

이론 2장 · 6

"CSS가 안 먹는다" 진단 절차

이 순서대로 확인하면 거의 모든 경우가 해결됩니다.

"CSS가 안 먹는다" 진단 절차 (1/2)

1 추측하지 말고 순서대로 확인하세요.

2 개발자도구 Elements → Styles를 연다.

내가 쓴 규칙이 목록에 보이는가? 안 보이면 CSS 파일이 연결되지 않았거나 선택자가 그 요소에 안 맞은 것입니다.

3 규칙에 취소선이 그어져 있는가?

그렇다면 진 것입니다. 바로 위에 이긴 규칙이 있으니 그것과 특정도·순서를 비교합니다.

4 Computed 탭에서 실제 적용된 값을 본다.

어디서 온 값인지 함께 표시됩니다.

"CSS가 안 먹는다" 진단 절차 (2/2)

- 1** **오타를 확인한다.**
colour, back-ground, 세미콜론 누락, 중괄호 짝. CSS는 틀린 줄을 조용히 무시합니다.
- 2** **그 속성이 그 요소에 먹는 속성인가?**
인라인 요소에 width, static 요소에 z-index는 안 먹습니다.
- 3** **캐시를 의심한다.**
강력 새로고침(Ctrl+Shift+R / Cmd+Shift+R).

6.1 특히 자주 나오는 세 가지

증상	원인	해결
미디어쿼리가 무시된다	뒤에 같은 특정도의 규칙이 있음	반응형 규칙을 파일 뒤로
z-index 가 안 먹는다	position 이 static	relative 등을 지정
인라인 요소에 width 가 안 먹는다	인라인은 크기를 못 가짐	display: inline-block 또는 block

이론 2장 · 7

규칙을 관리하는 방법

CSS는 파일이 커질수록 "무엇이 무엇을 덮는지" 알기 어려워집니다.

규칙을 관리하는 방법

- 1 실무에서 쓰는 관리 원칙 몇 가지입니다.

유지보수되는 CSS의 습관

● 정리

- 1 특정도를 낮게 유지한다.**
클래스 한두 개로 끝내고, 아이디로 스타일을 주지 않습니다.
- 2 베이스 → 레이아웃 → 컴포넌트 → 유틸리티 → 반응형**
순서로 파일을 배치합니다. 순서가 곧 우선순위입니다.
- 3 원본을 고치지 말고 덮어쓰기 층을 만든다.**
재사용한 베이스의 CSS를 직접 고치면 어떤 규칙이 어디서 왔는지 추적할 수 없게 됩니다. 새 파일을 만들어 맨 마지막에 불러오세요.
- 4 클래스 이름에 의미를 담는다.**
.red-text 가 아니라 .text-danger — 나중에 색을 바꿔도 이름이 거짓말을 하지 않습니다.
- 5 색·여백은 CSS 변수로 모은다.**
흩어진 값을 나중에 전수로 고치는 일은 반드시 실수를 부릅니다.

규칙을 관리하는 방법

- 1 베이스 CSS를 재사용할 때 반복해 밟는 함정 세 가지: ① `h1~h6 { color }` 가 어두운 배경 컴포넌트를 덮는다 ② `ul { list-style: none }` 이 본문 목록의 점까지 없앤다 ③ 색 토큰을...

웹 기술 이론 2장 마무리

2장 마무리

CSS — 캐스케이드와 특정도 — 본문과 도해는 학습사이트에서 다시 읽습니다

✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시

✓ 함께 볼 주차: 4주차 · 6주차 · 7주차

✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다